Torneo de Komikan Concurrente

Sistemas Operativos · Tarea 2 | Benjamín Oyarzún · Rodrigo Cantin

1. Diseño del esquema de sincronización

1.1 ¿Cómo está armado el sistema?

Separamos el sistema en procesos, para que la caída de un jugador no arrastre al resto:

```
Servidor  (hilos: clientes, matchmaker, cosechador, señales)
  └── Mesa / Juez   (un proceso por partida, máximo M a la vez)
        ├── Jugador Puma    (proceso con 1 hilo)
        └── Jugador Perros  (proceso con 12 hilos, uno por perro)
```

El tablero lo crea el Juez en **memoria compartida** (`mmap` anónimo) antes de hacer `fork` de los jugadores. Así los tres procesos ven exactamente el mismo tablero.

1.2 ¿Qué protege al tablero?

- **Un mutex por casilla** (28 en total). Cada casilla es un "slot" que las fichas compiten por tomar.
- **Un mutex de estado**, para los datos globales: capturas, perros vivos, resultado y registro de eventos.
- **Una variable de condición**, con la que el Juez duerme hasta que ocurre una jugada.

Todos los mutex son `PROCESS_SHARED` (funcionan entre procesos) y `ROBUST` (si su dueño muere, se pueden recuperar).

1.3 ¿Cómo se mueve una ficha sin pisar a otra?

Cada ficha repite este ciclo:

- **Mirar:** espera entre 50 y 300 ms y copia el tablero para decidir su jugada.
- **Reservar:** toma los locks de las casillas que va a usar (2 si es un paso, 3 si es una captura), siempre de menor a mayor índice.
- **Revisar de nuevo:** con los locks tomados, confirma que la casilla destino sigue libre. Mientras decidía, otra ficha pudo haber llegado.
- **Mover:** si todo sigue igual, se mueve, anota el evento y avisa al Juez.

Si dos perros quieren la misma casilla, gana el que toma primero su lock. El otro, al revisar, la encuentra ocupada y lo vuelve a intentar en su próximo ciclo. Las reglas de victoria se revisan en ese mismo momento, así que nunca hay un instante en que el Puma esté acorralado sin que el sistema lo note.

1.4 Servidor, caídas y Ctrl+C

- **Límite de M mesas:** un semáforo `sem_t` que parte en M. Se hace `sem_wait` antes de abrir una mesa y `sem_post` cuando termina. Los pares que sobran quedan dormidos esperando.
- **Caída de un jugador:** el Juez la detecta con `waitpid` y le da la victoria al oponente por abandono. Si el proceso murió con un lock tomado, el siguiente que lo pide recibe `EOWNERDEAD` y lo recupera. Al cerrar la mesa se desbloquea y destruye todo.
- **Ctrl+C:** un hilo dedicado recibe la señal, deja de aceptar jugadores y le pide a cada mesa (con `SIGTERM`) que cierre su partida. El servidor espera a todos los procesos y libera semáforo, mutex, condiciones y archivos.

2. Análisis de riesgos de concurrencia

Deadlock

Escenario: el perro A (en 8) quiere ir a 9 y el perro B (en 9) quiere ir a 8. A toma el 8 y espera el 9; B toma el 9 y espera el 8. Ninguno avanza.

Solución: todos piden los locks en el mismo orden, de menor a mayor índice, y el mutex de estado siempre al final. En el ejemplo, ambos piden primero el 8: uno espera al otro y no se forma un ciclo.

Lock huérfano

Escenario: un jugador muere mientras tiene un lock. Con un mutex normal, cualquiera que lo pida después queda bloqueado para siempre.

Solución: mutex robustos. Lo probamos matando a propósito al proceso Perros con `SIGKILL`: como mueren los 12 hilos a la vez, pueden quedar varios locks tomados, y se recuperan todos.

Livelock

Escenario: dos fichas que chocan, reintentan al mismo tiempo y vuelven a chocar una y otra vez.

Solución: no hay reintentos inmediatos. La ficha que pierde vuelve a esperar un tiempo aleatorio, así que las dos se desincronizan. Además, un límite de jugadas (`max_moves`) corta las partidas que no avanzan.

Starvation

Escenario: una ficha que pierde siempre la carrera por una casilla, o un par de clientes que nunca consigue mesa.

Solución: tomar un lock dura microsegundos y la espera entre jugadas es de 50 a 300 ms, así que perder siempre es prácticamente imposible. Los clientes se atienden en orden de llegada (cola FIFO).

Fork en un programa con hilos

Escenario: al hacer `fork`, el hijo hereda solo el hilo que lo llamó. Si otro hilo tenía un lock tomado, en la copia queda tomado para siempre.

Solución: solo el hilo matchmaker hace `fork`, y la mesa no usa ningún lock del servidor. El log se escribe con `write()`, en un bloque por llamada, para que no se mezclen las líneas de distintas mesas.

3. Evaluación

¿Por qué un mutex por casilla y no uno solo?

Nuestra primera versión usaba un solo mutex para todo el tablero. Era simple y sin riesgo de deadlock, pero las 13 fichas se movían de a una. Con un mutex por casilla, las fichas que están en zonas distintas se mueven en paralelo, y solo compiten cuando de verdad quieren la misma casilla. El costo es más complejidad (orden de bloqueo y revisión doble), que nos pareció justificado porque calza con la idea de "reclamar un slot".

¿Por qué procesos y no solo hilos?

Con procesos, una caída es real: el sistema operativo nos avisa con `waitpid` y la memoria del Juez no se ve afectada. El costo es usar primitivas compartidas entre procesos y tener más cuidado al limpiar.

Lo que medimos

- 200 clientes con M = 8: 100 partidas en unos 1,7 s, sin pasar nunca de 8 mesas activas.
- 350 partidas de estrés revisando el tablero en cada evento: 0 estados inválidos.
- Con los retardos reales (50 a 300 ms), una partida dura entre 1 y 5 segundos: el tiempo lo pone la espera del enunciado, no la sincronización.
- ThreadSanitizer no encontró condiciones de carrera. Después de un Ctrl+C, valgrind muestra 0 bytes en uso en el servidor y no quedan procesos ni memoria compartida sueltos.
- El lector de `config.txt` pasa 31 casos borde (valores vacíos, líneas incompletas, números inválidos, posiciones repetidas, etc.).

> **Sobre la semilla:** con la misma `seed`, cada ficha toma las mismas decisiones. Pero el orden en que se cruzan los hilos lo decide el planificador del sistema operativo, así que dos ejecuciones pueden terminar distinto. Eso es normal en un sistema concurrente real.